

Introduction to Python

Module 2 — Data Structures

Lists, dictionaries, tuples, and sets

Why We Need Data Structures

So far each variable held *one* value. But real data comes in **collections** — a list of documents, a table of counts, a set of unique words. Python has built-in structures for exactly this.

Structure	Holds	Example
list	ordered, changeable	["a", "b", "c"]
dict	key → value pairs	{"count": 5}
tuple	ordered, <i>unchangeable</i>	(52.1, 5.6)
set	unordered, <i>unique</i>	{"a", "b"}

Lists and dictionaries are the workhorses — you'll use them in almost every program.

Lists

A **list** is an ordered collection, written in square brackets. It can hold anything, and you can change it.

```
words = ["cat", "dog", "bird"]

len(words)    # 3 (how many items)
words[0]      # "cat" (indexing starts at 0!)
words[-1]     # "bird" (negative counts from the end)

words.append("fish")  # add to the end
words[0] = "lion"     # change an item
```

Indexing starts at 0, not 1 — the first item is `words[0]`. This trips up everyone at first.

Slicing Lists

Grab a **range** of items with `[start:stop]`. The stop index is *not* included.

```
nums = [10, 20, 30, 40, 50]

nums[1:3]  # [20, 30] (items 1 and 2, not 3)
nums[:2]   # [10, 20] (from the start)
nums[2:]   # [30, 40, 50] (to the end)
```

The same slicing works on **strings** too — a string is just a sequence of characters: `"hello"[1:4]` gives `"ell"`.

Dictionaries

A **dictionary** stores **key** → **value** pairs. Instead of position, you look things up by a *key*. Perfect for word counts, settings, records.

```
counts = {"cat": 3, "dog": 5}

counts["cat"]      # 3 (look up by key)
counts["bird"] = 2  # add a new pair
counts["cat"] = 4   # update a value

counts.keys()      # all keys
counts.values()     # all values
```

Keys are usually strings; values can be anything. Dictionaries are everywhere in data work.

Tuples & Sets (Briefly)

Tuple — like a list, but **frozen**

```
point = (52.1, 5.6)
```

- Round brackets
- **Cannot** be changed after creation
- Use for fixed pairs (coordinates, RGB)
- `point[0]` still works

Set — **unique** items, no order

```
tags = {"a", "b", "a"}
```

- Curly braces
- **Duplicates removed** automatically
- Great for “unique words”
- `set(my_list)` de-duplicates a list

Remember the “types vs. tokens” idea? `set(words)` turns a list of tokens into the set of unique types.

Module 2: Takeaways

- ✓ **Lists** [...]: ordered, changeable; index with [0], slice with [1:3]
- ✓ **Indexing starts at 0**; -1 is the last item
- ✓ **Dictionaries** {key: value}: look up by key, not position
- ✓ **Tuples** (...): like lists but unchangeable
- ✓ **Sets** {...}: unordered collections of unique items

Next: Module 3 — **control flow**: making decisions with `if` and repeating with loops.